DATA ENGINEERING

Apache Spark Visual Guide

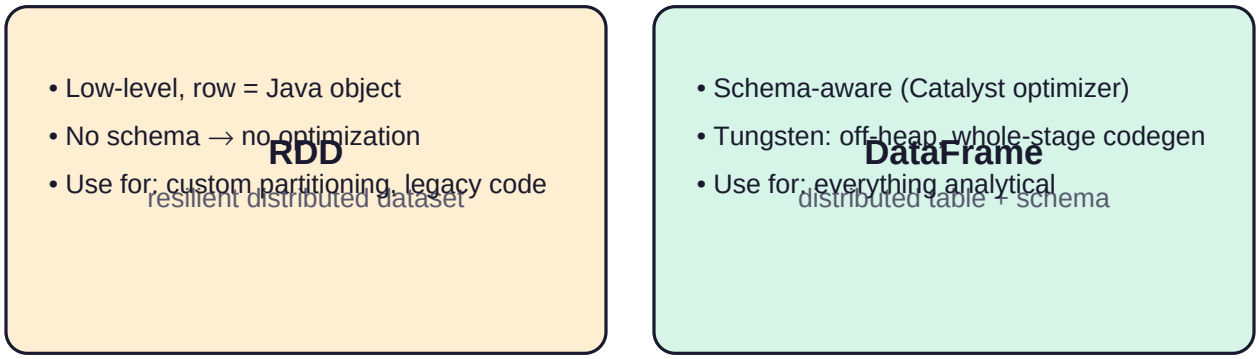
From RDDs to DataFrames, DAGs to shuffles.
How Spark turns your code into distributed execution.

ANIMATED WALKTHROUGH
Step through pages to play the animation

STEP 1/6

RDDs vs DataFrames

Two APIs, one engine. DataFrames win for almost everything today.



DataFrames let the Catalyst optimizer rewrite your query plan; RDDs execute exactly what you wrote. Datasets sit in between (typed + optimized).

STEP 2/6

Lazy Evaluation: Nothing Runs Until an Action

Transformations build a plan. Actions trigger execution.

Spark records transformations as a DAG and optimizes the whole plan before running anything — predicate pushdown, column pruning, join reordering.

- Transformations: map, filter, join — lazy, return new DataFrames
- Actions: count, collect, write — eager, trigger jobs
- Why lazy: the optimizer sees the full query before choosing a plan

STEP 3/6

DAG Scheduler: Stages & Tasks

Narrow transformations pipeline in one stage; shuffles split stages.

Narrow: each output partition needs one input partition (map, filter).

Wide: each output partition needs MANY inputs (groupBy, join) → shuffle.

One stage = one set of tasks, no data exchange between tasks.

The DAGScheduler cuts the plan at every shuffle boundary. Fewer shuffles = fewer stages = faster jobs.

STEP 4/6

The Shuffle: Spark's Expensive Habit

All-to-all data exchange across the network — the #1 performance killer.

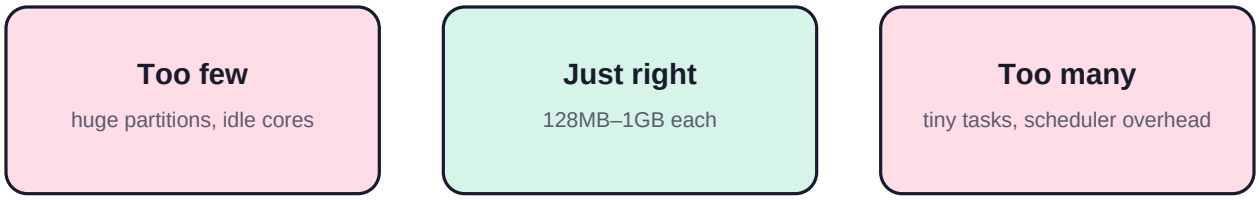
Every shuffle writes intermediate files to disk and pulls them across the network. Sort-merge join, repartition, and distinct all shuffle.

- Avoid: unnecessary groupBy, repartition on huge keys
- Prefer: broadcast joins for small tables (no shuffle)
- Tune: spark.sql.shuffle.partitions (default 200 is often wrong)

STEP 5/6

Partitioning: Data Layout Is Performance

Right-sized partitions keep every core busy without drowning in overhead.



Aim for partitions ~2–4x the number of cores. Skew (one giant partition) is worse than count — salt hot keys to break it up.

- repartition(n): full shuffle, use to increase parallelism
- coalesce(n): narrow, use to shrink partitions before writing
- Skew fix: salt the hot key (key + random suffix), then de-salt

STEP 6/6

Tuning Checklist

The highest-leverage knobs, in order.

- Cache reused DataFrames (.persist) — but unpersist when done
- Broadcast small tables (<10MB default) to kill shuffle joins
- Partition pruning: filter on the partition column first
- Avoid UDFs — they block Catalyst optimization; use native functions
- Write fewer, larger files: coalesce before writing to storage

Measure first: the Spark UI's SQL tab shows which stage shuffles and how long each takes — tune the red stages, not the green ones.

STEP ✓I✓

Key Takeaways

Apache Spark — Remember

- DataFrames beat RDDs: schema-aware plans get Catalyst optimization and whole-stage codegen.
- Nothing runs until an action — laziness lets the optimizer see and rewrite the full plan.
- The DAG splits into stages at shuffle boundaries; narrow transforms pipeline, wide ones shuffle.
- Shuffles are the dominant cost: all-to-all network + disk I/O. Design them away with broadcast joins.
- Right-size partitions (128MB–1GB), salt skewed keys, and let the Spark UI tell you where time goes.

Keep exploring

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev